

LLM-D

Kubernetes-Native Distributed LLM Inference

Architecture, Implementation & Production Engineering Notes

March 2026

Document Scope

This reference document covers llm-d from first principles through production deployment. The document covers the full stack: architecture decisions, component internals, KV-cache optimization, disaggregated serving, multi-GPU parallelism, autoscaling, observability, and real-world deployment patterns.

1. System Overview

1.1 What is LLM-D?

llm-d (pronounced "LLM-dee") is an open-source, Kubernetes-native, high-performance distributed LLM inference serving framework. It was created to close the gap between raw model server throughput and the operational demands of production-grade serving at scale. The project is led by engineers from Red Hat, Google, IBM, and the broader open-source community.

At its core, llm-d is an orchestration layer that sits above model servers such as vLLM and SGLang, providing intelligent routing, KV-cache-aware scheduling, disaggregated prefill/decode serving, tiered cache offloading, and SLO-driven autoscaling — all expressed as composable, Helm-deployable components on Kubernetes.

⚡ Critical Distinction

llm-d is NOT an agent framework. It does not manage reasoning loops, tool use, or memory systems. llm-d is an inference serving control plane — analogous to what a Kubernetes network fabric is to application pods. Its job is to route tokens to the right GPU at the right time with minimal latency and maximum hardware utilization.

1.2 Core Philosophy & Design Goals

- Fastest "time to SOTA performance" — validated, reproducible configurations for frontier models
- Hardware agnosticism — NVIDIA A100/H100/B200, AMD MI250, Google TPU v5e+, Intel Gaudi2/3
- Open standards composition — Kubernetes Gateway API, Envoy proxy, OpenTelemetry, vLLM upstream
- Separation of concerns — model execution (vLLM) vs orchestration (llm-d) vs infrastructure (K8s)
- Production-first design — observability, resilience, and reproducibility are first-class constraints
- "Well-lit paths" — opinionated, tested recipes for common serving topologies

1.3 Problems llm-d Solves vs. Traditional LLM Pipelines

Problem	Traditional Approach	llm-d Solution
KV-cache waste across replicas	Each pod recomputes identical prefixes	KV-cache indexer routes requests to pods holding matching cache blocks
Prefill starvation of decode	Monolithic servers mix prefill + decode, causing jitter	P/D disaggregation: dedicated prefill and decode worker pools
GPU memory as cache ceiling	Cache bounded by HBM; recompute on eviction	Hierarchical offload: GPU → CPU DRAM → NFS/Lustre/IBM Storage Scale
Cold starts on scale-up	New replicas must warm up independently	Shared filesystem cache; new replicas hydrate immediately from global store
Naive round-robin load balancing	Random routing ignores cache state → redundant work	EPP scoring: routes to the pod with highest cache-hit probability

Problem	Traditional Approach	Ilm-d Solution
Multi-adapter thundering herd	All replicas load all LoRA adapters	LoRA-precise prefix routing: adapters co-located with their cache blocks
No SLO-aware scaling	HPA on CPU/GPU utilization; misses latency signals	WVA autoscaler uses queue depth + KV pressure + traffic mix
Network tail latency in disaggregation	UCX under congestion degrades sharply	UCCL backend: host-side transport stack, 2.4× more resilient

1.4 Key Abstractions

Model Server (vLLM)

The execution engine for transformer inference. vLLM manages attention, KV-cache blocks, and token generation via PagedAttention. Ilm-d contributes and maintains upstream vLLM changes for disaggregated serving, KV-connector interfaces, and production observability.

InferencePool (Kubernetes CRD)

A Kubernetes custom resource that groups a set of vLLM pods serving the same model. InferencePools are the scheduling unit — the EPP routes requests to pods within a pool. Defined via the Kubernetes Gateway API Inference Extension (GIE/IGW).

Endpoint Picker Plugin (EPP)

The scheduling brain of Ilm-d. EPP is an Envoy external-processing extension that intercepts each request and uses filter+scorer plugins to select the optimal vLLM pod. EPP is the primary extensibility point for custom routing policies.

KV-Cache Indexer

A real-time, distributed index of which KV-cache blocks reside on which pods (and at which tier: GPU/CPU/disk). It consumes KVEvents from vLLM pods and answers scoring queries from EPP: "which pod has the most blocks for this prompt?"

NIXL / UCCL Transport

The KV-transfer layer between prefill and decode pods. NIXL provides a unified abstraction over IB/RoCE RDMA, TPU ICI, and DCN. UCCL (Unified Collective Communication Library) provides a host-resident transport stack with adaptive congestion control on top of NIXL.

ModelService Helm Chart

An opinionated Helm chart for declaring and deploying vLLM workloads. It abstracts parallelism settings (TP, DP), accelerator allocation (device-plugin or DRA), and routing sidecar configuration into clean YAML values.

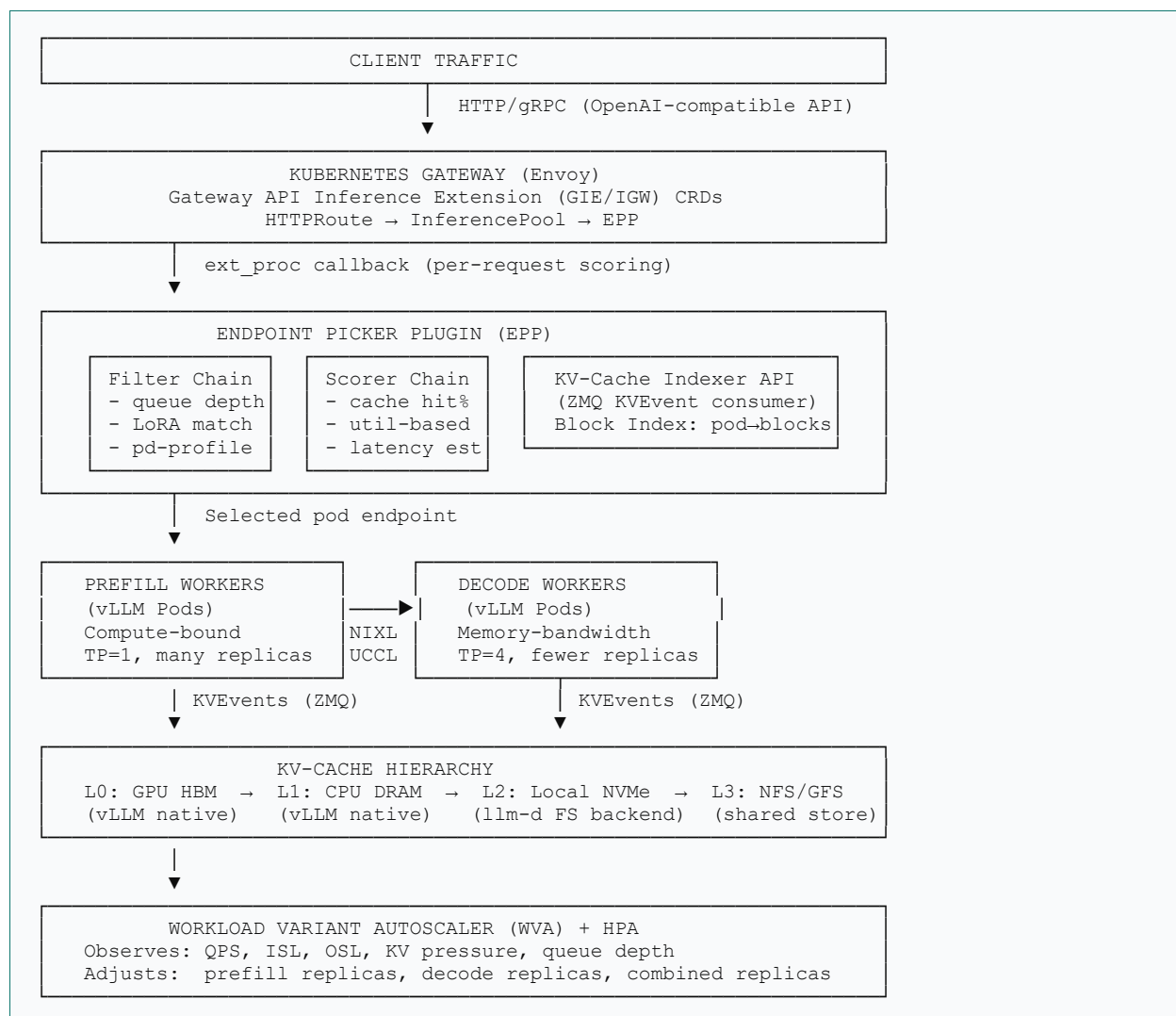
Workload Variant Autoscaler (WVA)

A traffic- and hardware-aware autoscaler that models the capacity of each instance variant (prefill-only, decode-only, combined), measures incoming traffic mix (QPS, ISL/OSL distribution, QoS tier), and calculates the optimal replica mix to maintain SLOs at minimum cost.

2. Architecture Deep Dive

2.1 Full System Architecture

llm-d integrates three well-defined layers: an infrastructure layer (Kubernetes), a control/data plane (Gateway API Inference Extension + Envoy), and an inference execution layer (vLLM). The diagram below shows the canonical single-model topology:



2.2 Control Plane vs. Data Plane

Control Plane

- **Kubernetes API Server:** source of truth for InferencePool, HTTPRoute, pod lifecycle events
- **EPP:** stateful process embedding filter/scorer plugins; subscribes to pod lifecycle via K8s informers
- **KV-Cache Indexer:** listens for ZMQ KVEvents from vLLM; maintains block-level locality index

- **WVA**: control loop that reads inference metrics and adjusts replica counts via K8s scale API
- **Helmfile / Helm Charts**: declarative deployment configuration for the entire stack

Data Plane

- **Envoy Proxy**: handles all request traffic; delegates routing decisions to EPP via ext_proc gRPC
- **vLLM Model Servers**: execute forward passes, manage PagedAttention KV blocks, stream tokens
- **NIXL / UCCL**: transfers KV-cache tensors from prefill to decode nodes over RDMA or TCP-X
- **llm-d FS Backend**: async I/O layer connecting vLLM KVConnector interface to NFS/Lustre/Storage Scale

2.3 Request Lifecycle (End-to-End)

```

Step 1  Client sends OpenAI-compatible POST /v1/chat/completions to Gateway IP
      ↓
Step 2  Envoy Gateway receives request; HTTPRoute matches → InferencePool
      Envoy calls EPP via ext_proc gRPC (request_headers / request_body phase)
      ↓
Step 3  EPP Filter Chain runs:
      a. Queue-depth filter  - exclude pods with queue > threshold
      b. LoRA filter          - if request has adapter, exclude pods missing it
      c. PD-profile filter    - decide if request needs P/D or direct-to-decode
      ↓
Step 4  EPP Scorer Chain runs (on filtered pod set):
      a. KV-cache scorer     - query KV-Indexer for cache-hit scores per pod
      b. Utilization scorer  - factor in current KV-cache utilization %
      c. Predicted-latency scorer (experimental) - estimate TTFT per pod
      Composite score → select winning pod
      ↓
Step 5  EPP returns routing header to Envoy → Envoy proxies to selected pod
      ↓
Step 6a [P/D path] Envoy routes to Prefill worker
      Prefill worker processes full prompt → generates KV blocks
      P/D sidecar on Decode worker coordinates KV transfer via NIXL/UCCL
      KV tensors transferred over IB/RoCE to Decode worker GPU HBM
      ↓
Step 6b [Direct decode path] Request goes directly to Decode/combined worker
      Worker checks local KV-cache (GPU HBM → CPU DRAM → NVMe) for prefix hits
      Computes only uncached prefix, then autoregressive decoding begins
      ↓
Step 7  Tokens stream back through Envoy to client (SSE / chunked transfer)
      ↓
Step 8  vLLM emits KVEvents (block created/evicted) via ZMQ
      KV-Cache Indexer updates block locality index
      EPP learns cache state for next routing decision

```

2.4 Comparison: llm-d vs. Alternatives

Capability	llm-d	Raw vLLM (standalone)	Ray Serve	Triton IS	TorchServe
KV-cache-aware routing	☑ Native (EPP + Indexer)	✗ None	✗ None	✗ None	✗ None
P/D disaggregation	☑ NIXL/UCCL, multi-HW	☑ Experimental	✗ None	✗ None	✗ None

Capability	llm-d	Raw vLLM (standalone)	Ray Serve	Triton IS	TorchServe
Tiered KV offload	☑ GPU→CPU→NFS (async)	☑ CPU only (limited)	✗ None	✗ None	✗ None
LoRA-aware scheduling	☑ Precise cache routing	✗ None	✗ None	✗ None	✗ None
Scale-to-zero	☑ v0.5 (activator pattern)	✗ None	⚠ Limited	✗ None	✗ None
Wide Expert Parallelism	☑ EP=16+, MoE (DeepSeek)	☑ Experimental	⚠ Manual	✗ None	✗ None
K8s native control plane	☑ Gateway API, CRDs	✗ None	⚠ Via Ray on K8s	⚠ Limited	⚠ Limited
Multi-HW accelerators	☑ NVIDIA/AMD/TPU/Gaudi	☑ Same	⚠ NVIDIA focus	☑ NVIDIA/CPU	⚠ NVIDIA focus
Reproducible benchmarks	☑ Per-guide benchmark CLI	⚠ Manual	⚠ Manual	⚠ Manual	⚠ Manual

🔍 Design Insight: Why Not Just Use vLLM Directly?

vLLM is a single-node inference engine with excellent per-GPU efficiency. What it lacks is cluster-level intelligence.

When you run 8 vLLM pods behind a naive K8s Service (round-robin), each pod independently manages its own KV-cache. A request with a system prompt already cached on Pod 3 may route to Pod 7, which recomputes from scratch.

At scale, this cache fragmentation becomes the primary latency driver — not compute.

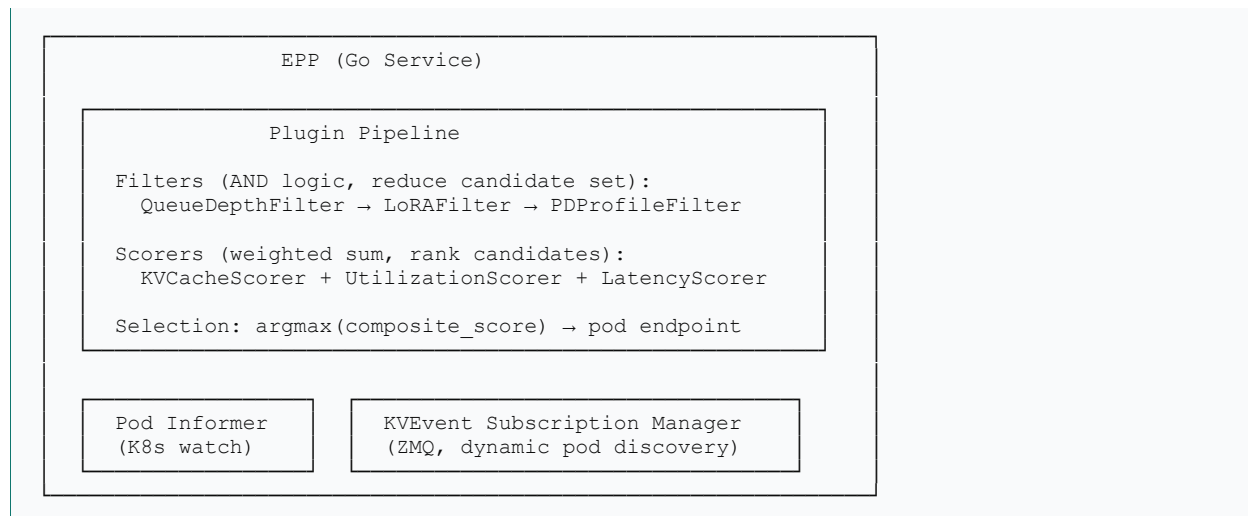
llm-d adds the routing intelligence that transforms 8 isolated pods into a coherent cache-aware serving cluster.

3. Core Components

3.1 Inference Scheduler (EPP — Endpoint Picker Plugin)

The EPP is a Go service that implements the Envoy ext_proc gRPC protocol. It is the primary scheduling brain for llm-d.

Architecture



Key Interfaces

- Filter interface: `Filter(ctx, candidates []Pod, req *Request) []Pod`
- Scorer interface: `Score(ctx, pod Pod, req *Request) float64`
- Balancer interface: `Balance(ctx, scoredPods []ScoredPod) Pod`
- Plugin registration: plugins declared in YAML config, loaded at startup

Built-in Plugins

- **pd-profile-handler**: inspects prompt length vs. threshold to decide prefill/decode routing path; configurable threshold enables "Selective PD" (route short prompts directly to decode)
- **kv-cache-scorer**: calls KV-Cache Indexer with tokenized prompt hash; returns per-pod hit-rate score in `[0, 1]`
- **utilization-filter**: filters pods where `running_queue / max_queue > threshold` (prevents overloading)
- **lora-aware-filter**: for LoRA requests, filters to pods that have loaded the requested adapter and hold matching cache blocks
- **predicted-latency-scorer (experimental)**: models expected TTFT and TPOT per pod based on queue state; replaces queue-depth heuristics with latency-model estimates

⚠ EPP Statefulness

EPP is a stateful service — it maintains in-memory pod state and the KV-block index.

For high availability, v0.5 introduced Active-Active multi-replica EPP with dynamic ZMQ subscription

management. Each EPP replica independently discovers pods and updates its local index. Ensure at least 2 EPP replicas in production with anti-affinity rules.

3.2 KV-Cache Indexer

A high-performance library (Go) that maintains a global, near-real-time view of KV-cache block locality across the vLLM fleet. It is embedded within EPP and also exposed as a standalone scoreable API.

Data Model

```
// Block key: hash of token sequence up to block boundary
type BlockKey struct {
    Hash      uint64    // rolling hash of token IDs in this block
    ModelID   string    // model namespace/id
    AdapterID string    // LoRA adapter ID (" " for base)
}

// Block Index entry
type BlockEntry struct {
    PodID      string    // Kubernetes pod name
    Tier       Tier      // GPU | CPU | DISK
    CreatedAt  int64     // Unix nanoseconds
    EvictedAt  int64     // 0 if still resident
}

// Index: BlockKey → []BlockEntry (multi-pod awareness)
type KVBlockIndex map[BlockKey][]BlockEntry
```

Write Path (Event Ingestion)

vLLM emits KVEvents via ZMQ publish socket when blocks are created or evicted. The Event Subscriber in EPP consumes these events and updates the in-memory KVBlockIndex.

```
// KVEvent (from vLLM)
type KVEvent struct {
    EventType string    // "block_stored" | "block_evicted"
    RequestID string
    BlockKeys []uint64  // hashed token sequences
    PodID     string
    Tier      string    // "gpu" | "cpu" | "disk"
    Timestamp int64
}
```

Read Path (Scoring)

On each request, EPP tokenizes the prompt, computes rolling block hashes (configurable block size, typically 16–64 tokens), queries the index, and returns per-pod hit counts normalized to [0, 1].

```
// Scoring query
func (idx *KVBlockIndex) ScorePods(
    promptTokens []int,
    candidates   []string, // pod IDs
    adapterID    string,
) map[string]float64 {
    blocks := computeBlockKeys(promptTokens, blockSize)
    scores := make(map[string]float64)
    for _, pod := range candidates {
        hits := 0
        for _, bk := range blocks {
            if entries := idx[bk]; hasPod(entries, pod) {
                hits++
            }
        }
        scores[pod] = float64(hits) / float64(len(blocks))
    }
    return scores
}
```

3.3 Model Service (llm-d-modelservice)

A Helm chart that provides a declarative interface for deploying vLLM-based model servers. It is the primary user-facing abstraction for configuring inference workloads in llm-d.

Key Configuration Surface

Field	Description	Example
modelArtifacts.uri	Model source (HuggingFace, PVC, OCI)	hf://meta-llama/Llama-3.1-70B-Instruct
decode.parallelism.tensor	Tensor parallel size for decode pods	4
decode.parallelism.data	Data parallel size (replica count)	2
prefill.replicas	Number of prefill worker pods	4
accelerator.type	GPU vendor (nvidia / amd / google / intel-gaudi)	nvidia
accelerator.dra	Use K8s Dynamic Resource Allocation	true
multinode	Use LeaderWorkerSet for multi-node TP	true
routing.proxy.image	P/D sidecar image (routing-sidecar)	ghcr.io/llm-d/llm-d-routing-sidecar:latest

Parallelism Auto-Calculation

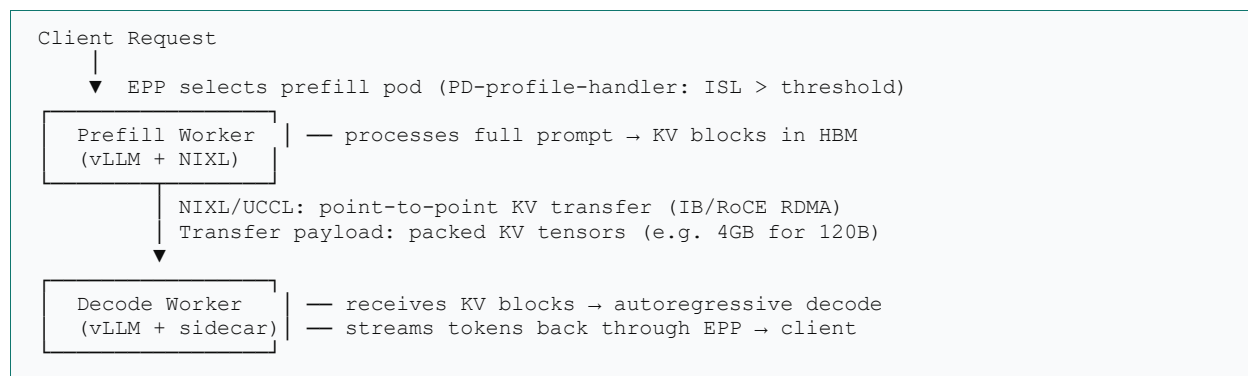
The chart automatically calculates accelerator resource requests: $\text{nvidia.com/gpu} = \text{tensor} \times \text{dataLocal}$. Users can override by setting `resources.limits` explicitly (required for TPUs where $\text{TP} \neq \text{device count}$).

3.4 Prefill / Decode Disaggregation (P/D)

P/D disaggregation splits the two phases of LLM inference onto specialized worker pools:

- **Prefill Workers:** compute-bound; process the full input prompt; optimized for high arithmetic intensity; typically TP=1 with many replicas
- **Decode Workers:** memory-bandwidth-bound; autoregressive token generation; typically higher TP to increase KV-cache capacity

P/D Transaction Flow



When to Use P/D Disaggregation

- **Use P/D when:** model is 70B+ parameters, input sequence length (ISL) > 4K tokens, MoE architecture with wide expert parallelism, or ITL SLO is strict
- **Skip P/D when:** model is ≤ 20B parameters, ISL < 1K tokens, network bandwidth is limited (TCP-only, < 100Gb/s)
- **Selective PD:** configure pd-profile-handler threshold to route short prompts directly to decode, avoiding transfer overhead for requests where prefill is cheaper than transfer latency

3.5 Tiered KV-Cache Architecture

llm-d implements a three-tier memory hierarchy for KV-cache storage, decoupling cache capacity from local GPU HBM:

TIER 0: GPU HBM (fastest, most expensive) - PagedAttention blocks, directly used during compute - Capacity: 40-80 GB per GPU - vLLM native management
TIER 1: CPU DRAM (fast, cheaper) - vLLM native CPU offload (pin_memory=True) - Capacity: 512GB-2TB per node - Access latency: ~10-50μs
TIER 2: Local NVMe (medium speed, large capacity) - llm-d FS backend via vLLM KVConnector - Capacity: 4-30 TB per node - Access latency: ~100-500μs
TIER 3: Shared Filesystem (NFS / Lustre / IBM Storage Scale) - Global cross-replica store; new replicas hydrate from here

- Capacity: Petabytes; shared across cluster
- Access latency: 1-10ms (HPC FS can achieve <2ms)

llm-d FS Backend Design

- **Interface:** implements vLLM KVConnector abstract class — standard upstream interface
- **Async I/O:** fully non-blocking; parallel worker threads handle block fetch/store without blocking decode loop
- **Block addressing:** deterministic path: {fs_root}/{model_id}/{block_hash}.kv — enables cross-replica sharing
- **Eviction policy:** LRU at GPU tier; write-through to lower tiers; pluggable eviction policies via KVConnector config

Performance Impact of Tiered Caching

Benchmark: Llama-3.1-70B, 4×H100, 16K token requests with repeated prompts

GPU-only: throughput collapses at ~50 concurrent users (HBM saturation)

GPU+FS: sustains ~185K tok/s at 250 users — 13.9× improvement

Key insight: the value is not peak throughput but throughput floor stability under load.

A system that degrades gracefully is operationally superior to one that collapses.

4. Implementation Details

4.1 Deploying a Minimal llm-d Stack

The minimal production-viable llm-d deployment consists of: Kubernetes cluster, Gateway (Istio or kgateway), llm-d-infra (EPP + KV-Cache Indexer), and a ModelService deployment. Below is a step-by-step implementation guide.

Step 1: Prerequisites

```
# Kubernetes 1.29+, kubectl, helm 3.x, helmfile
# GPU nodes with device plugin installed
# HuggingFace token for model download

# Install helmfile
$ brew install helmfile # or download from GitHub releases

# Clone llm-d
$ git clone https://github.com/llm-d/llm-d.git
$ cd llm-d

# Set HuggingFace token secret
$ kubectl create secret generic llm-d-hf-token \
  --from-literal=HF_TOKEN=<your_token> -n llm-d
```

Step 2: Install Infrastructure (Gateway + EPP)

```
# Deploy llm-d-infra: Gateway provider + EPP + monitoring stack
$ cd guides/inference-scheduling
$ export NAMESPACE=llm-d
$ kubectl create namespace ${NAMESPACE}

# Deploy with Istio gateway (or use -e kgateway for kgateway)
$ helmfile apply -n ${NAMESPACE}

# Verify EPP is running
$ kubectl get pods -n ${NAMESPACE} | grep epp
gaie-is-epp-54444ddc66-qv6ds 1/1 Running 0 2m

# Verify Envoy gateway is running
$ kubectl get pods -n ${NAMESPACE} | grep gateway
infra-is-inference-gateway-istio-56d66db57f-zwtzn 1/1 Running 0 2m
```

Step 3: Deploy a Model Service

```
# Install ModelService Helm chart
$ helm repo add llm-d-modelservice \
```

```

https://llm-d-incubation.github.io/llm-d-modelservice/
$ helm repo update

# Deploy Llama-3.1-8B with TP=2, 2 replicas
$ helm install llama8b llm-d-modelservice \
  --namespace ${NAMESPACE} \
  --set modelArtifacts.name="meta-llama/Llama-3.1-8B-Instruct" \
  --set modelArtifacts.uri="hf://meta-llama/Llama-3.1-8B-Instruct" \
  --set modelArtifacts.authSecretName=llm-d-hf-token \
  --set decode.replicas=2 \
  --set decode.parallelism.tensor=2 \
  --set accelerator.type=nvidia

```

Step 4: Install HTTPRoute

```

# Connect Gateway → InferencePool
$ kubectl apply -f httproute.yaml -n ${NAMESPACE}

# Get gateway IP
$ export GATEWAY_IP=$(kubectl get gateway/llm-d-inference-gateway \
  -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')

# Test inference
$ curl http://${GATEWAY_IP}/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "meta-llama/Llama-3.1-8B-Instruct",
    "messages": [{"role": "user", "content": "Hello!"}]}'

```

4.2 P/D Disaggregation Deployment

```

# Deploy GPT-OSS-120B with P/D: 4 prefill (TP=1) + 1 decode (TP=4)
$ cd guides/pd-disaggregation
$ export NAMESPACE=llm-d-pd
$ kubectl create namespace ${NAMESPACE}

# Deploy the full P/D stack
$ helmfile apply -n ${NAMESPACE}

# Expected pods after deployment:
# gaie-pd-epp-*          1/1 Running (EPP)
# infra-pd-inference-gateway-* 1/1 Running (Envoy)
# ms-pd-llm-d-modelservice-decode-* 2/2 Running (vLLM + sidecar)
# ms-pd-llm-d-modelservice-prefill-* 1/1 Running (*4 replicas)

```

4.3 Tiered KV-Cache with FS Backend

```
# values-tiered-cache.yaml (ModelService override)
decode:
  containers:
    - name: vllm
      args:
        # Enable vLLM KVConnector interface
        - "--kv-connector=llmd_fs_backend"
        - "--kv-connector-extra-config={
          \"fs_root\": \"/mnt/kv-cache\",
          \"async_workers\": 4,
          \"pin_memory\": true
        }"
        - "--cpu-offload-gb=32" # CPU DRAM tier size

# Mount shared filesystem (NFS example)
volumes:
  - name: kv-cache-fs
    nfs:
      server: "nfs-server.example.com"
      path: "/exports/kv-cache"
volumeMounts:
  - name: kv-cache-fs
    mountPath: /mnt/kv-cache
```

4.4 Wide Expert Parallelism (MoE Models)

For Mixture-of-Experts models like DeepSeek-R1/V3 and Mixtral, Wide Expert Parallelism (Wide-EP) enables massive throughput by spreading expert computation across many GPUs.

```
# Wide-EP with LeaderWorkerSet: EP=16, DP=16, TP=1
$ cd guides/wide-ep-lws
$ helmfile apply -n ${NAMESPACE}

# Corresponding ModelService values (wide-ep):
multinode: true # Use LeaderWorkerSet
decode:
  parallelism:
    tensor: 1 # TP=1 per GPU
    data: 16 # DP=16 across pods
    dataLocal: 1 # local DP per node
    workers: 16 # 16 workers per LWS group
    replicas: 16

# Validated result (B200 cluster):
```

```
# Topology: 16 prefill × 16 decode (EP=16, DP=16)
# Result:   ~50K output tokens/sec total
#           ~3.1K output tokens/sec per decode GPU
```

4.5 Custom EPP Plugin Development

Advanced users can implement custom Filter or Scorer plugins in Go and register them with the EPP plugin framework.

```
// Custom scorer: prefer pods in same availability zone as client
package myplugins

import (
    "context"
    "github.com/llm-d/llm-d-inference-scheduler/pkg/scheduling"
)

type AZAffinityScorer struct {
    clientAZ string
}

func (s *AZAffinityScorer) Score(
    ctx context.Context,
    pod scheduling.Pod,
    req *scheduling.Request,
) float64 {
    podAZ := pod.Labels["topology.kubernetes.io/zone"]
    if podAZ == s.clientAZ {
        return 1.0 // maximum affinity score
    }
    return 0.0
}

// Register in plugin_config.yaml:
// plugins:
//   - type: az-affinity-scorer
//     parameters:
//       weight: 0.3
```

4.6 Building an llm-d-like Intelligent Router (Python Reference)

For teams building custom inference routers, the following Python reference implementation illustrates the core EPP scoring logic outside the Kubernetes context:

```
import hashlib
import asyncio
```

```

from dataclasses import dataclass, field
from typing import List, Dict, Optional
from collections import defaultdict

# — Data Structures —————

@dataclass
class KVBlock:
    key: int          # hash of token subsequence
    pod_id: str
    tier: str          # "gpu" | "cpu" | "disk"
    timestamp: float

@dataclass
class Pod:
    id: str
    queue_depth: int = 0
    kv_utilization: float = 0.0 # 0.0 - 1.0
    lora_adapters: List[str] = field(default_factory=list)

# — KV Block Index —————

class KVBlockIndex:
    def __init__(self, block_size: int = 32):
        self.block_size = block_size
        # block_hash → {pod_id: KVBlock}
        self._index: Dict[int, Dict[str, KVBlock]] = defaultdict(dict)

    def _compute_block_keys(self, tokens: List[int]) -> List[int]:
        """Rolling hash of token sequences, one hash per block."""
        keys = []
        for i in range(0, len(tokens), self.block_size):
            block = tokens[i:i + self.block_size]
            h = int(hashlib.sha256(
                bytes([t % 256 for t in block])
            ).hexdigest(), 16) & 0xFFFFFFFF
            keys.append(h)
        return keys

    def ingest_event(self, event_type: str, pod_id: str,
                    block_keys: List[int], tier: str = "gpu"):
        """Handle KVEvent from vLLM (block_stored / block_evicted)."""
        import time
        for key in block_keys:
            if event_type == "block_stored":

```

```

        self._index[key][pod_id] = KVBlock(key, pod_id, tier,
                                             time.time())

    elif event_type == "block_evicted":
        self._index[key].pop(pod_id, None)

def score_pods(
    self,
    tokens: List[int],
    candidates: List[str],
) -> Dict[str, float]:
    """Return cache-hit score in [0,1] per candidate pod."""
    keys = self._compute_block_keys(tokens)
    if not keys:
        return {p: 0.0 for p in candidates}
    scores = {}
    for pod_id in candidates:
        hits = sum(1 for k in keys if pod_id in self._index.get(k, {}))
        scores[pod_id] = hits / len(keys)
    return scores

# — Scheduler (EPP logic) —————

class InferenceScheduler:
    def __init__(
        self,
        kv_index: KVBlockIndex,
        queue_threshold: int = 32,
        kv_weight: float = 0.7,
        util_weight: float = 0.3,
    ):
        self.kv_index = kv_index
        self.queue_threshold = queue_threshold
        self.kv_weight = kv_weight
        self.util_weight = util_weight

    def select_pod(
        self,
        pods: List[Pod],
        tokens: List[int],
        lora_adapter: Optional[str] = None,
    ) -> Optional[Pod]:
        # Step 1: Filter — remove overloaded pods
        candidates = [
            p for p in pods
            if p.queue_depth < self.queue_threshold

```

```

    ]
    # Step 2: Filter — LoRA match if required
    if lora_adapter:
        candidates = [
            p for p in candidates
            if lora_adapter in p.lora_adapters
        ]
    if not candidates:
        return None

    # Step 3: Score — KV-cache hit rate
    pod_ids = [p.id for p in candidates]
    kv_scores = self.kv_index.score_pods(tokens, pod_ids)

    # Step 4: Score — utilization penalty (lower util = better)
    util_scores = {p.id: 1.0 - p.kv_utilization for p in candidates}

    # Step 5: Composite score
    composite = {
        p.id: (self.kv_weight * kv_scores[p.id]
              + self.util_weight * util_scores[p.id])
        for p in candidates
    }

    # Step 6: Select argmax
    best_id = max(composite, key=composite.get)
    return next(p for p in candidates if p.id == best_id)

# — Example Usage —————

index = KVBlockIndex(block_size=32)
scheduler = InferenceScheduler(kv_index=index)

# Simulate KVEvent from pod "vllm-pod-0" storing prefix blocks
system_prompt_tokens = list(range(256)) # 256 token system prompt
block_keys = index._compute_block_keys(system_prompt_tokens)
index.ingest_event("block_stored", "vllm-pod-0", block_keys)

# New request with same system prompt
pods = [Pod("vllm-pod-0", queue_depth=2), Pod("vllm-pod-1", queue_depth=1)]
selected = scheduler.select_pod(pods, tokens=system_prompt_tokens)
print(f"Selected: {selected.id}") # → "vllm-pod-0" (100% cache hit)

```

5. Optimization Techniques

5.1 Inference Optimizations

5.1.1 KV-Cache Strategies

Strategy	Mechanism	When to Use	Impact
Prefix Cache (GPU)	vLLM PagedAttention reuses blocks for matching prefixes	Always — zero cost	Up to 80% TTFT reduction for shared system prompts
CPU Offload (Tier 1)	vLLM <code>--cpu-offload-gb</code> : evicted GPU blocks spill to pinned DRAM	GPU HBM constrained workloads	Extends effective cache 4–10×
Filesystem Offload (Tier 2/3)	llm-d FS backend: async write to NVMe/NFS via KVConnector	High-concurrency, long contexts, multi-tenant SaaS	13.9× throughput floor under load
KV-Cache-Aware Routing	EPP routes to pod with highest block hit rate	Always with EPP enabled	Prevents redundant prefill across replicas
LoRA-Precise Routing	EPP routes LoRA requests to pods holding adapter + matching blocks	Multi-tenant LoRA workloads	Eliminates thundering herd; reduces per-request overhead

5.1.2 Batching Strategies

- **Continuous Batching (vLLM default):** new requests join the batch at iteration boundaries without waiting for current batch to complete; maximizes GPU utilization; critical for decode-heavy workloads
- **Chunked Prefill:** limits prefill token budget per iteration (`--max-num-batched-tokens`) to prevent decode starvation; set to 512–2048 tokens for interactive workloads
- **Speculative Decoding:** small draft model proposes N tokens; target model verifies in single forward pass; useful for decode-bound workloads; configured in vLLM with `--speculative-model` and `--num-speculative-tokens`; v0.4 integration with P/D architecture
- **Wide-EP Batching:** for MoE models, EP=16+ enables parallel expert routing across GPUs; maximizes throughput for batch processing use cases where latency constraints are relaxed

5.1.3 Parallel Tool Execution

While llm-d is an inference serving layer (not an agent framework), applications that call llm-d can implement parallel tool execution. The serving layer itself can be optimized for tool-use patterns:

- **Stateless requests:** tool-result injection creates new requests with expanded context; prefix caching covers shared system prompt and prior conversation turns
- **Context management:** structure tool-use conversations so system prompt + N turns are a stable prefix; variable parts (tool results) are always appended — maximizing cache hit rate
- **Request parallelism:** parallel tool calls generate multiple simultaneous completion requests; llm-d handles them efficiently through continuous batching

5.2 System-Level Optimizations

5.2.1 Latency vs. Quality Tradeoffs

Lever	Low Latency Config	High Quality / Throughput Config
TP size	Smaller TP (more replicas)	Larger TP (higher VRAM per model copy)
ISL routing	Direct-to-decode (no P/D transfer latency)	P/D disaggregation (better ITL at high concurrency)
Speculative decoding	N=4–8 draft tokens (high acceptance rate)	Off (add latency variance)
KV offload tier	GPU-only (deterministic latency)	CPU+FS offload (higher throughput floor)
Queue threshold	Low (fewer requests inflight, lower P99)	High (maximize utilization, worse tail latency)

5.2.2 Token Efficiency Strategies

- **Prompt templating:** standardize system prompt format across tenants to maximize prefix cache hit rate; a 1K-token system prompt cached on all pods saves 1K tokens of compute per request
- **KV block alignment:** align prompt boundaries to block_size (default 32 tokens); misaligned prompts waste partial blocks and reduce cache hit rate
- **Context window management:** for long conversations, use sliding window or summarization to keep active context within L0 (GPU HBM) capacity; tiered offload helps but GPU-resident context is still fastest
- **Model quantization:** FP8 weight quantization halves VRAM consumption, doubling KV-cache space per GPU; vLLM supports FP8 on H100/H200; enable with `--dtype fp8` or `--quantization fp8`

5.2.3 Cost Optimization Patterns

- **Scale-to-zero:** v0.5 WVA supports scale-down to 0 replicas during idle periods; activator component holds requests during cold start; critical for dev/test and infrequent RAG workloads
- **Saturation-based scaling:** WVA triggers scale-out based on queue depth + KV-cache pressure (not just QPS); prevents latency degradation before SLO violations occur
- **Mixed instance types:** WVA models capacity per instance type; route latency-tolerant (batch) requests to cheaper/older GPUs; serve interactive requests from premium (H100/B200) instances
- **Preemptive scaling:** traffic forecasting + predictive scaling (roadmap: v0.6) will enable proactive provisioning before demand spikes rather than reactive healing

5.3 Multi-GPU and Distributed Setup

5.3.1 Parallelism Strategies

```

TENSOR PARALLELISM (TP):
- Each GPU holds 1/TP fraction of model weights
- All-reduce communication per attention layer
- Best for: single-node, NVLink-connected GPUs
- Reduces memory per GPU; increases per-request throughput
- Configure: decode.parallelism.tensor = 2|4|8

DATA PARALLELISM (DP) / Replicas:
- Multiple full model copies; each handles independent requests
- No cross-pod communication during inference
- Best for: scaling throughput with sufficient GPU memory
- Configure: decode.replicas = N (or decode.parallelism.data)

```

```

EXPERT PARALLELISM (EP):
- For MoE: distribute experts across GPUs
- EP=16 → 16 GPUs each hold 1/16 of experts
- All-to-all communication for expert routing
- Requires fast interconnect (IB/NVLink) for efficiency
- Configure: via vLLM --enable-expert-parallelism flag in args

PIPELINE PARALLELISM (PP):
- Layers distributed across GPUs sequentially
- Currently managed by vLLM internally
- Not a primary llm-d configuration surface (use TP instead)

```

5.3.2 Network Requirements by Parallelism Mode

Mode	Communication Pattern	Min Bandwidth	Recommended Fabric
TP (single node)	All-reduce (NVLink)	600 GB/s+	NVLink 4.0 (H100 SXM)
TP (multi-node)	All-reduce (IB/RoCE)	200–400 Gb/s	InfiniBand NDR or RoCE 200G
EP (MoE)	All-to-all (IB/RoCE)	200–400 Gb/s	InfiniBand NDR or RoCE 200G
P/D transfer	Point-to-point (NIXL)	200 Gb/s+	InfiniBand or RoCE (RDMA); TCP fallback
DP replicas	None (independent)	Ethernet (for API)	Standard Kubernetes networking

5.3.3 vLLM Configuration for Production

```

# vLLM args for a production 70B model serving pod (H100×4, TP=4)
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --tensor-parallel-size 4 \
  --dtype bfloat16 \
  --max-model-len 131072 \
  --max-num-batched-tokens 2048 \      # chunked prefill
  --max-num-seqs 256 \
  --gpu-memory-utilization 0.90 \
  --enable-prefix-caching \           # critical for llm-d routing
  --cpu-offload-gb 32 \               # CPU tier offload
  --kv-connector llmd_fs_backend \    # filesystem tier
  --kv-connector-extra-config '{...}' \
  --disable-log-requests \
  --uvicorn-log-level warning

# Note: --enable-prefix-caching MUST be set for EPP KV routing to work.
# Without it, vLLM does not track block hashes and EPP scores are useless.

```

6. Workflow & Execution Model

6.1 Standard Request Lifecycle

Below is the complete step-by-step lifecycle for a typical interactive chat completion request through an llm-d cluster.

Phase 1: Ingress and Routing

- Client sends POST /v1/chat/completions with JSON payload
- Envoy Gateway receives request; HTTPRoute matches to InferencePool by model header/path
- Envoy invokes EPP via ext_proc gRPC (ProcessingRequest: request_headers phase)
- EPP parses request: extracts model ID, prompt, LoRA adapter header if present
- EPP retrieves current pod state from K8s informer cache (queue depth, utilization)

Phase 2: Scheduling Decision

- EPP Filter Chain runs sequentially; each filter narrows the candidate pod set
- KV-Cache Indexer tokenizes prompt and computes rolling block hashes
- Indexer queries in-memory KVBlockIndex for per-pod hit counts
- Scorer Chain produces weighted composite score per candidate
- EPP returns routing decision header to Envoy (x-envoy-upstream-rq-timeout-ms + pod endpoint)

Phase 3: Inference Execution

- **[Direct path]:** Envoy forwards request to selected vLLM pod; vLLM checks local prefix cache (GPU → CPU → NVMe); computes uncached prefix tokens; begins autoregressive decoding
- **[P/D path]:** Envoy routes to prefill worker; prefill worker processes full prompt context; P/D sidecar on decode worker receives KV block transfer params via NIXL; KV tensors transfer over RDMA; decode worker begins token generation from received KV state

Phase 4: Streaming Response

- vLLM streams tokens via Server-Sent Events (SSE) or chunked transfer encoding
- Envoy streams response back to client with minimal buffering
- Each generated KV block emits a KVEvent (block_stored) via ZMQ
- KV-Cache Indexer consumes events and updates block locality index in real-time

Phase 5: Post-Request Updates

- When context evicted from GPU, vLLM emits KVEvent (block_evicted); Indexer removes entries
- If CPU/disk offload enabled, blocks survive eviction from GPU — Indexer updates tier field
- Prometheus metrics updated: queue depth, TTFT, ITL, KV utilization, cache hit rate

6.2 Failure Handling and Retries

Pod Failure

- K8s watches detect pod eviction; EPP informer immediately removes pod from candidate set
- In-flight requests to failed pod: Envoy circuit breaker triggers; request retried to next-best pod
- KV-cache blocks on failed pod: Indexer removes entries via pod lifecycle event; next request for same prompt re-prefills on surviving pod (or retrieves from shared FS tier if available)

EPP Failure (Active-Active HA)

- v0.5 supports multi-replica EPP; Envoy load-balances ext_proc calls across EPP replicas
- Each EPP replica maintains independent state; ZMQ subscription manager reconnects dynamically
- Routing quality degrades gracefully (one replica has stale index) but correctness is maintained

Overload and Backpressure

- Queue threshold filter prevents sending requests to pods above capacity ceiling
- Envoy flow control mechanisms (max_pending_requests, max_requests) provide additional backpressure
- WVA detects queue pressure metrics and triggers scale-out before P99 SLO violation

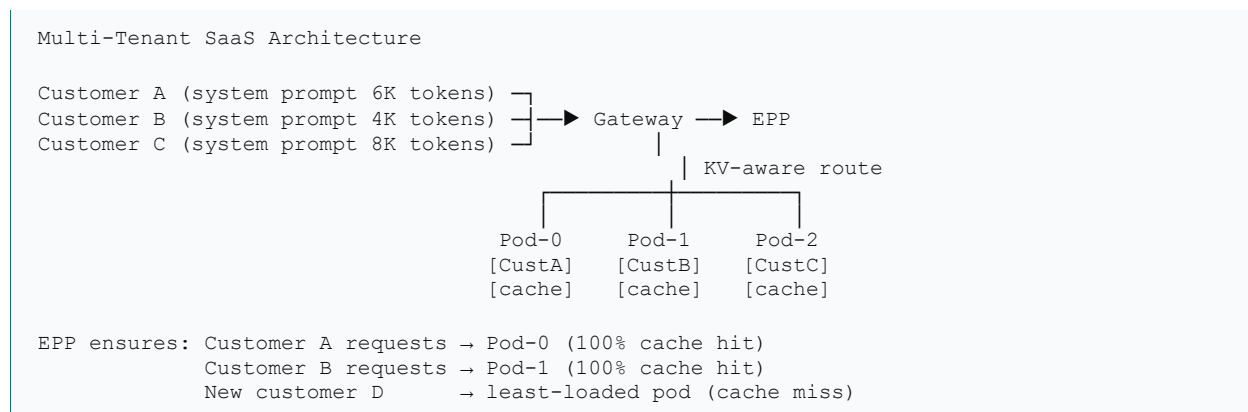
6.3 Guardrails and Safety

- llm-d is a serving control plane — content-level safety (toxicity filtering, PII detection) must be implemented in the application layer or as vLLM sampling parameters
- SLO enforcement: predicted-latency scorer (experimental) enables routing away from pods near latency SLO boundary
- Rate limiting: implement upstream at Envoy Gateway layer using standard Envoy rate limit filters before requests reach EPP
- Model access control: HTTPRoute header matching enables per-model routing; authentication implemented at Gateway layer via JWT/OIDC filters

7. Real-World Use Cases

7.1 Multi-Tenant SaaS LLM Service

Architecture: Multiple customer organizations share a GPU cluster. Each customer has a unique system prompt. High cache hit rate is the primary cost driver.

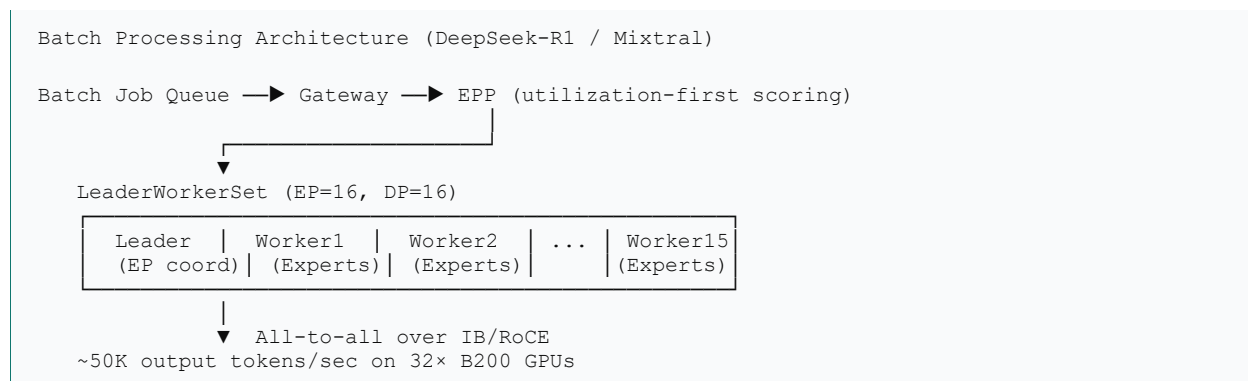


Performance result: 8×vLLM pods, 16×H100 GPUs, Qwen3-32B, shared 6K prefix:

- **Throughput:** 4.5–11K output tokens/sec (vs. 2–5K with round-robin)
- **P50 TTFT:** 136–157ms vs. 3–8s with round-robin under load
- **TTFT improvement:** 99% reduction at high QPS
- **Throughput improvement:** 109% higher than K8s round-robin baseline

7.2 Large-Model Batch Processing (Wide-EP)

Use case: RL training data generation, offline summarization pipelines, document processing. Throughput maximized; latency constraints relaxed.



Critical configuration for Wide-EP:

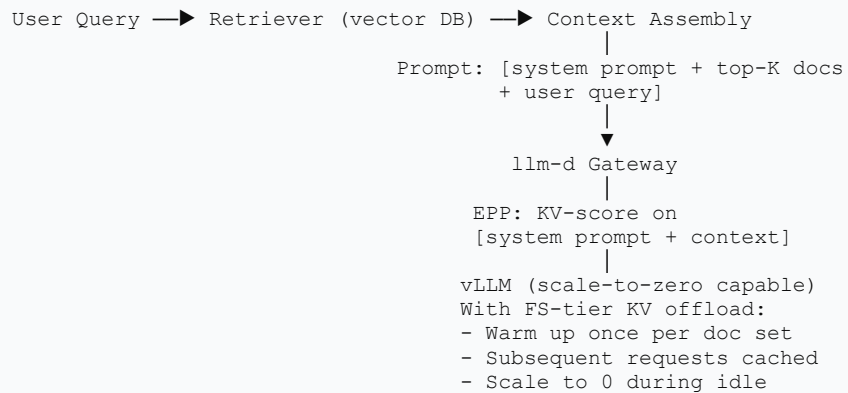
- **Interconnect:** RDMA mandatory; TCP-X as fallback with additional tuning
- **Queue threshold:** raise significantly for batch workloads (queue_depth up to 512+)
- **Chunked prefill:** set max-num-batched-tokens higher (4096+) for batch; not interactive

- **UCCL**: enable for resilience under cross-traffic congestion in shared clusters

7.3 RAG Application with Shared Document Index

Use case: Internal RAG application over a shared document corpus. System prompt contains retrieved context (2–10K tokens). Cold starts are infrequent. Scale-to-zero ideal.

RAG Application Architecture



Key optimization: structure RAG prompts so retrieved docs are always at the same position in the prompt — ensuring they become stable KV-cache prefix blocks that persist on the FS tier across idle periods.

8. Comparison & Design Insights

8.1 Ilm-d vs. Alternative Serving Stacks

Dimension	Ilm-d	Ray Serve	BentoML	Triton IS	OpenAI Assistants
Primary abstraction	Inference serving control plane (K8s native)	Python-native distributed serving	ML service packaging	NVIDIA-focused model server	Cloud-hosted API service
KV-cache intelligence	First-class (EPP + Indexer)	Not included	Not included	Not included	Black box
P/D disaggregation	Native (NIXL/UCCL)	Not included	Not included	Not included	N/A
Heterogeneous HW	NVIDIA / AMD / TPU / Gaudi	NVIDIA focus	NVIDIA focus	NVIDIA / ARM	N/A (managed)
Control plane	Kubernetes Gateway API	Ray cluster	BentoCloud / K8s	Kubernetes/Docker	Hosted (no control)
Operational maturity	Production (v0.5, IBM/Red Hat/Google)	Production (Anyscale)	Production (BentoCloud)	Production (NVIDIA)	N/A (managed)

8.2 When to Use Ilm-d

- ☑ Use Ilm-d when:** serving models in Kubernetes; running shared system prompts at scale; deploying 70B+ models with P/D requirements; building multi-tenant inference infrastructure; requiring hardware-agnostic serving across NVIDIA/AMD/TPU; needing reproducible, benchmarked serving configurations
- ⚠ Consider alternatives when:** serving small models (<7B) on single GPU without shared prefixes (raw vLLM simpler); building a Python-native data science platform (Ray Serve more ergonomic); running on managed cloud inference (latency SLOs met without complexity); team has no Kubernetes expertise (BentoCloud abstracts this)
- ✗ Do not use Ilm-d when:** looking for an agent orchestration framework (use LangGraph, CrewAI); building agent memory systems; implementing tool-calling pipelines; running in non-Kubernetes environments without adaptation

Design Insight: The "Brilliant Friend" vs. "Infrastructure" Framing

Ilm-d exists in the same layer as Istio or Envoy — it is networking infrastructure for tokens.

Just as you would not build Istio to create a web app, you should not build Ilm-d to make one API call.

Ilm-d becomes essential when: (1) you have many replicas, (2) requests share common prefixes, (3) you serve traffic from multiple tenants or use cases on shared GPU infrastructure.

The ROI of Ilm-d scales with cluster size and request prefix overlap.

8.3 Architecture Decision Guide

Question	Answer	Recommendation
How many GPU nodes?	1–2 nodes	Raw vLLM; no need for llm-d orchestration
How many GPU nodes?	3–50 nodes	llm-d inference scheduling (EPP + KV indexer)
How many GPU nodes?	50+ nodes	llm-d full stack (P/D + Wide-EP + WVA + tiered KV)
System prompt size?	< 512 tokens	Prefix caching in vLLM; llm-d routing optional
System prompt size?	512–8K tokens	KV-aware routing critical; llm-d EPP required
System prompt size?	8K+ tokens	FS-tier offload + P/D disaggregation recommended
Model size?	< 20B params	DP replicas; no P/D needed
Model size?	70B–120B params	P/D disaggregation; TP=4–8 per decode pod
Model size?	200B+ MoE params	Wide-EP (EP=16+); P/D; LeaderWorkerSet deployment
Adapter usage?	None	Standard EPP; no LoRA-specific config needed
Adapter usage?	Multiple LoRA	LoRA-precise prefix routing; cache-aware adapter pinning

9. Observability, Advanced Topics & Roadmap

9.1 Monitoring & Metrics

Prometheus Metrics

llm-d exposes metrics from multiple layers. Below are the key metrics to monitor in production:

Metric	Source	Description	Alert Threshold
vllm:num_requests_running	vLLM	Active requests in model engine	> max_num_seqs × 0.9
vllm:gpu_cache_usage_perc	vLLM	GPU KV-cache block utilization	> 0.85
vllm:time_to_first_token_seconds	vLLM	TTFT histogram (P50/P95/P99)	P99 > SLO
vllm:time_per_output_token_seconds	vLLM	ITL histogram (P50/P95/P99)	P99 > SLO
kv_cache_hit_rate	EPP / KV Indexer	Prefix cache hit rate across fleet	< 0.5 (investigate routing)
epp_routing_latency_ms	EPP	Time EPP takes to select a pod	> 10ms P99
pd_transfer_latency_ms	P/D sidecar	KV tensor transfer latency (prefill→decode)	> 500ms P99
inference_pool_queue_depth	GIE/EPP	Pending requests per InferencePool	> threshold × 0.8

Grafana Dashboards

- **llm-d Fleet Overview:** per-pod TTFT, ITL, throughput, KV utilization; cache hit rate by model and adapter
- **P/D Disaggregation:** KV transfer latency distribution; prefill vs decode queue depth balance; UCCL vs UCX latency comparison
- **KV-Cache Hierarchy:** block counts per tier (GPU/CPU/disk); eviction rates; FS backend I/O latency
- **Autoscaler:** replica count over time; WVA scaling decisions with rationale; cost efficiency metrics (tokens per dollar)

9.2 Distributed Tracing

OpenTelemetry tracing is implemented across the full serving path in llm-d: Gateway → EPP → vLLM engine.

- Gateway spans: request arrival, routing decision, response completion
- EPP spans: filter chain execution, scorer execution, index query latency
- vLLM spans: queue wait, prefill computation, decode iterations, KV cache operations
- P/D spans: KV transfer initiation, NIXL/UCCL transport, decode start

Roadmap (v0.6): end-to-end distributed traces with granular per-layer breakdown to expose bottlenecks in disaggregated architectures.

9.3 Advanced Scheduling Patterns

Predicted Latency-Based Routing (Experimental)

Rather than scoring pods on queue depth alone, the predicted-latency scorer models expected TTFT/TPOT per pod based on current queue state, KV utilization, and request ISL/OSL characteristics. This enables SLO-targeted routing: avoid pods that would violate the P99 TTFT target, even if their queue appears manageable.

LoRA Adapter Cache Pinning

In v0.5, the scheduler tracks LoRA adapter locality separately from base model KV blocks. When a LoRA request arrives, EPP filters to pods that have loaded the requested adapter AND hold matching KV blocks for the request prefix. This prevents the thundering herd pattern where every pod loads every adapter on request.

Active-Active EPP HA

Multiple EPP replicas subscribe independently to ZMQ KVEvent streams from all vLLM pods. Each EPP maintains its own KV-block index. Envoy load-balances ext_proc calls across EPP replicas. Under failure, surviving replicas serve with potentially stale (but not incorrect) routing data.

9.4 Security Considerations

- **Network isolation:** vLLM pods should be in a separate K8s namespace with NetworkPolicy restricting access to Gateway and EPP only; external traffic never reaches pods directly
- **Model access control:** HTTPRoute matching and Gateway-level JWT/OIDC authentication; model-level RBAC via InferencePool labels
- **KV-cache confidentiality:** cached KV blocks can contain sensitive conversation content; filesystem-tier blocks persisted to shared NFS require appropriate filesystem-level access controls and encryption at rest
- **Rate limiting:** implement at Gateway (Envoy RateLimitFilter) before requests reach EPP; prevents DoS exhausting GPU queue
- **Tenant isolation:** for strict multi-tenant isolation, use separate InferencePools per tenant; shared pools improve efficiency but mix KV-cache blocks across tenants

9.5 Roadmap (v0.6+)

Area	Feature	Status
Scheduling	Predictive latency-aware routing (TTFT/TPOT targets vs queue depth)	v0.6 planned
Scheduling	Native batch inference support (offline processing mode)	v0.6 planned

Area	Feature	Status
Offloading	Proactive KV state management: pin critical context blocks	v0.6 planned
Offloading	Semantic caching policies (approximate prefix matching)	Research/incubation
Autoscaling	ML-based predictive scaling (queuing theory + traffic models)	v0.6 planned
Autoscaling	Direct SLO integration (TTFT/ITL targets drive scale decisions)	v0.6 planned
Observability	End-to-end distributed tracing: Gateway → EPP → engine	v0.6 planned
Transport	Further UCCL optimization for ultra-low tail latency	Ongoing
Hardware	AMD MI300X, Intel Gaudi3, Google TPU v6 validation	Ongoing

Final Design Insight: The Cache is the Architecture

The central insight driving all of llm-d's design choices is this:

In a multi-GPU LLM serving cluster, the KV-cache IS the primary performance variable.

Everything in llm-d is designed to maximize KV-cache utility:

- EPP routes to the pod with the highest cache hit rate
- P/D disaggregation isolates prefill computation so decode cache is not evicted
- Tiered offload extends cache beyond GPU HBM limits
- FS-tier shared storage makes cache a cluster-wide resource, not per-pod
- WVA autoscaler uses KV pressure as a scaling signal
- UCCL reduces transfer latency so cache can be reused across nodes cheaply

An ML engineer who understands KV-cache management IS an llm-d architect.

Appendix: Quick Reference

A.1 Component Repository Map

Component	Repository	Language
Core / Guides / Helm	github.com/llm-d/llm-d	YAML / Shell
Inference Scheduler (EPP)	github.com/llm-d/llm-d-inference-scheduler	Go
KV-Cache Indexer	github.com/llm-d/llm-d-kv-cache	Go
Model Service (Helm chart)	github.com/llm-d-incubation/llm-d-modelservice	YAML / Helm
Infrastructure (Helm)	github.com/llm-d/llm-d-infra	YAML / Helm
Inference Simulator	github.com/llm-d/llm-d-inference-sim	Go / Python

Component	Repository	Language
Benchmark Tools	github.com/llm-d/llm-d-benchmark	Python
Workload Variant Autoscaler	github.com/llm-d-incubation/ig-wva	Go
Routing Sidecar	github.com/llm-d/llm-d-routing-sidecar	Go
KV Connectors / FS Backend	github.com/llm-d/llm-d-kv-cache (kv_connectors/)	Python / Go

A.2 Key Configuration Knobs Cheat Sheet

Knob	Where	Default	Production Recommendation
--enable-prefix-caching	vLLM args	false	Always enable — required for EPP routing
--cpu-offload-gb	vLLM args	0	16–64 GB per node ($\geq$ L1 DRAM headroom)
--max-num-batched-tokens	vLLM args	4096	512–2048 for interactive; 4096+ for batch
--gpu-memory-utilization	vLLM args	0.90	0.88–0.92 (leave headroom for fragmentation)
queue_threshold (filter)	EPP plugin config	32	Set to max_num_seqs $\times$ 0.85
kv_weight (scorer)	EPP plugin config	0.7	0.6–0.8; increase for shared-prefix workloads
pd threshold (selective PD)	EPP pd-profile-handler	0	Set ISL in tokens (e.g. 2048 for 2K+ $\rightarrow$ P/D)
decode.parallelism.tensor	ModelService values	1	2 for 7B, 4 for 70B, 8 for 120B+
prefill.replicas	ModelService values	1	3–5 $\times$ decode replicas for compute-bound workloads
accelerator.dra	ModelService values	false	Enable on K8s 1.32+ for flexible GPU allocation

A.3 Performance Benchmarks (v0.5 Validated)

Topology	Model	Hardware	Result
Wide-EP (EP=16, DP=16)	DeepSeek / Qwen MoE	32 $\times$ NVIDIA B200	~50K output tok/s; ~3.1K tok/s per decode GPU
Inference Scheduling	Qwen3-32B	16 $\times$ NVIDIA H100 (TP=2)	4.5–11K tok/s; P50 TTFT 136–157ms; 109% vs baseline
P/D Disaggregation	GPT-OSS-120B	16 $\times$ NVIDIA H200 (8P + 8D)	~26K total tok/s; P50 TTFT ~700ms; 0 failures at 1.2K ISL
Tiered KV Cache	Llama-3.1-70B	4 $\times$ H100 + IBM Storage Scale	185K tok/s at 250 users; 13.9 $\times$ vs GPU-only at peak load

Topology	Model	Hardware	Result
UCCL vs UCX resilience	Llama-3.1-8B	8× H100, 200Gb/s IB	UCCL: +7.1% latency under congestion vs UCX: +17.1%

— End of Document —